✅ Using ConfigMap for Environment Variables (NGINX Env Demo)

Instead of hardcoding values inside the Deployment, we move them into a **ConfigMap**. This allows:

- ✅ Runtime configuration without rebuilding images
- ✅ Environment-specific values (Dev, QA, Prod)
- ✅ Safer configuration management
- ✅ Easy updates without redeploying containers

✅ 1. ConfigMap Design for This Lab

We will externalize these two variables:

Variable Name	Purpose
WELCOME_MSG	Dynamic welcome message
ENV	Environment name (Dev/Stage/Prod)

✅ 2. Create ConfigMap YAML

📄 **File Name:** *nginx-env-demo-configmap.yaml*

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-env-demo-config
data:
  WELCOME_MSG: "Hello from Kubernetes using ConfigMap!"
  ENV: "Staging"
```

✅ This replaces the hardcoded **env:** values in your Deployment.

✅ 3. Updated Deployment YAML (Pulling from ConfigMap)

📄 **File Name:** *nginx-env-demo-deployment.yaml*

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-env-demo
  labels:
    app: nginx-env-demo
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx-env-demo
  template:
    metadata:
      labels:
        app: nginx-env-demo
    spec:
      containers:
        - name: nginx
          image: mydockerid/nginx-env-demo:v1
          ports:
            - containerPort: 80
          env:
            - name: WELCOME_MSG
              valueFrom:
                configMapKeyRef:
                  name: nginx-env-demo-config
                  key: WELCOME_MSG
            - name: ENV
              valueFrom:
                configMapKeyRef:
                  name: nginx-env-demo-config
                  key: ENV
```

✅ Now the pod dynamically reads values from the ConfigMap.

✅ **4. Apply ConfigMap and Deployment**

kubectl apply -f nginx-env-demo-configmap.yaml

kubectl apply -f nginx-env-demo-deployment.yaml

kubectl apply -f nginx-env-demo-service.yaml

✅ **5. Verify ConfigMap Injection**

🔹 **Check ConfigMap**

$ kubectl get configmap nginx-env-demo-config -o yaml

🔹 **Check Environment Variables Inside Pod**

$ kubectl get pods -l app=nginx-env-demo

$ kubectl exec -it <pod-name> -- printenv | grep -E "WELCOME_MSG|ENV"

✅ Expected Output:

WELCOME_MSG=Hello from Kubernetes using ConfigMap!

ENV=Staging

✅ **6. Test in Browser**

http://<node-ip>:30090

✅ You should see:

Hello from Kubernetes using ConfigMap!

Environment: Staging

✅ **7. Update ConfigMap Without Rebuilding Image**

Modify the ConfigMap:

data:

WELCOME_MSG: "Updated message without redeploying image"

ENV: "Production"

Apply:

$ kubectl apply -f nginx-env-demo-configmap.yaml

✅ Then restart pods to reload values:

$ kubectl rollout restart deployment nginx-env-demo

$ kubectl rollout status deployment/nginx-env-demo

✅ **8. Production Best Practices**

Best Practice	**Why**
Use ConfigMaps for non-sensitive data	Avoid rebuilding containers
Use Secrets for passwords & tokens	Security compliance
Keep ConfigMaps per environment	Dev / QA / Prod
Restart pods after ConfigMap updates	Ensure reload
Label ConfigMaps	For easy tracking